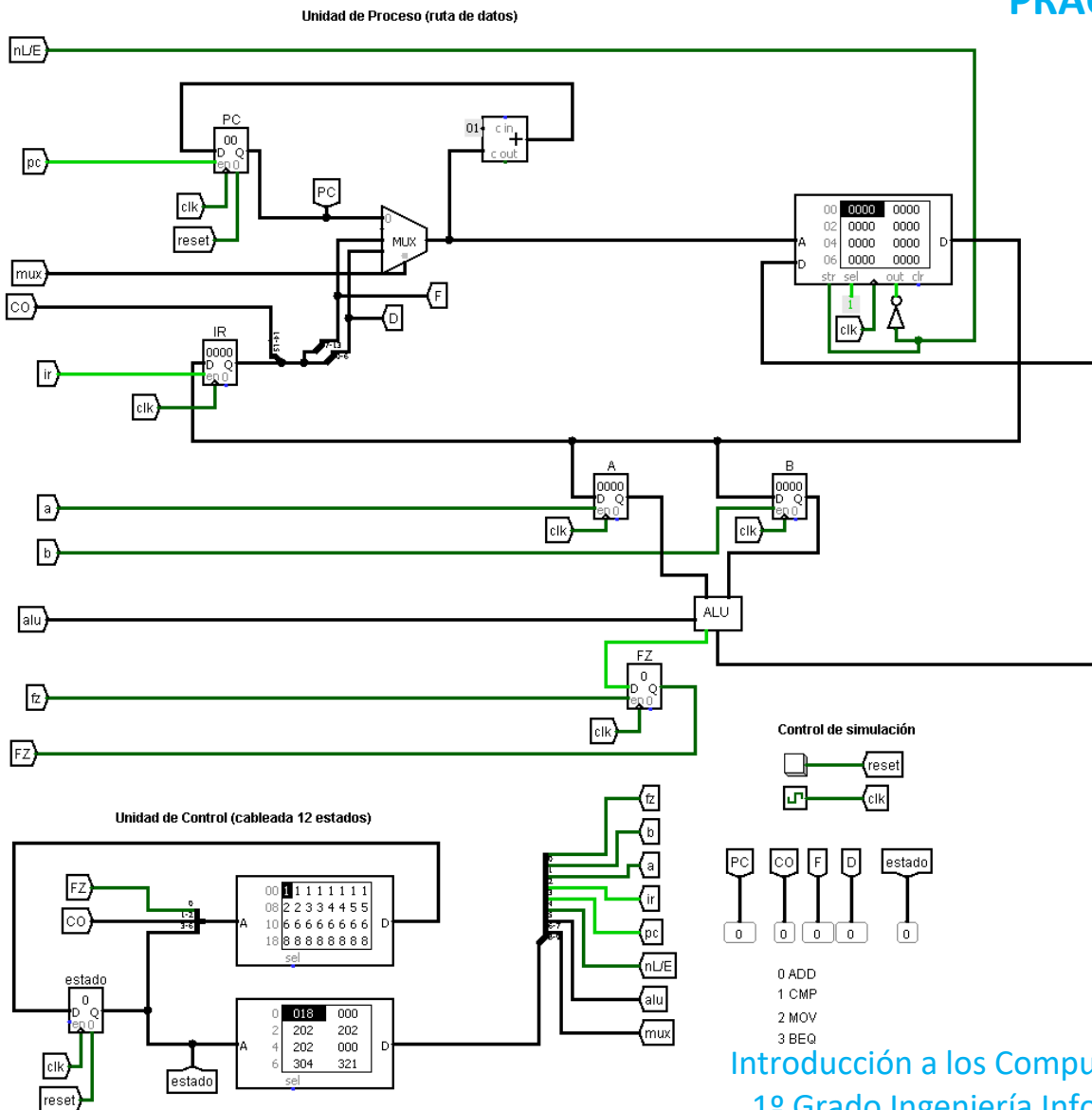


PROGRAMACIÓN Y DEPURACIÓN DE MÁQUINA SENCILLA

PRÁCTICA 6



Natalia Ayuso Luis M. Ramos Juan Segarra Víctor Viñals
 nayuso@unizar.es luisma@unizar.es jsegarra@unizar.es victor@unizar.es



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

1 RESUMEN

En esta práctica vamos a utilizar una implementación Logisim de Máquina Sencilla (MS). Programaremos y depuraremos para este procesador, ejecutando ciclo a ciclo varios programas con la unidad de control cableada de 12 estados. El objetivo principal es comprender el proceso de traducción desde un lenguaje de alto nivel (lenguaje C en este caso) hasta binario y entender perfectamente la ejecución de programas en MS.

Para poder realizar la práctica hay que llevar resuelto el Estudio Previo del Apartado 2.1. Al llegar al laboratorio coloca encima de la mesa el Estudio Previo de forma visible.

La práctica finaliza cuando los circuitos funcionan correctamente y **se han enviado los resultados de los apartados 2.2 y 3.2 a través de Moodle.**

2 UN PROGRAMA SENCILLO

2.1 TRABAJO PREVIO: COMPILACIÓN DEL PROGRAMA (C → ENSAMBLADOR → BINARIO)

El programa en **lenguaje C** que queremos ejecutar en MS es el siguiente:

```
int a=3;
int b=4;
int c;    // vamos a calcular c=a*b
int i;    // variable auxiliar de control del bucle

c=0;
i=0;
while (i!=b){
    c=c+a;
    i=i+1;
}
halt();
```

- a) Completa el programa equivalente traducido a **lenguaje ensamblador** (faltan las instrucciones entre las etiquetas WHILE y HALT).

```
.data 16 ; inicio de datos @16 (decimal)
a:      .dw 3 ; int a=3; .dw reserva una palabra y le asigna valor
b:      .dw 4 ; int b=4;
c:      .rw 1 ; int c; .rw 1 reserva una palabra sin inicializar
i:      .rw 1 ; int i;
cero:   .dw 0 ;
uno:    .dw 1 ;

.code ; inicio de código @0
    mov cero, c ; c=0;
    mov cero, i ; i=0;
WHILE: ; while (i!=b){
    ;
    ; c=c+a;
    ; i=i+1;
    ; }
    ;
HALT:   cmp cero, cero; halt();
    beq HALT ;
.end
```

- b) Traduce el programa en ensamblador a binario, y obtén el contenido de cada posición de memoria (especificado en hexadecimal).

2.2 TRABAJO EN EL LABORATORIO: SIMULACIÓN Y DEPURACIÓN

- Abre tu circuito de la práctica 4. Nómbralo como componente **ALU** (en el menú de componentes clic en **main** y escribir nuevo nombre en *tabla de propiedades / Nombre Del Circuito*). Guárdalo en un archivo nuevo (*Archivo / Guardar Como*).
- Añade un componente nuevo. Nómbralo **main**. Márcalo como circuito principal (*Proyecto / Seleccionar Como Circuito Principal*).
- Descarga *MS.circ* de Moodle y ábrelo desde Logisim (*Archivo / Abrir*).
- Selecciona todo (*Editar / Seleccionar Todo*), cópialo (*Editar / Copiar*) y pégalo (*Editar / Pegar*) en tu hoja **main**.
- Identifica todos los componentes de la unidad de proceso.
- Coloca y conecta tu **ALU** de la práctica 4. Puedes cambiar la posición de los pines de la **ALU** en *Proyecto / Edit Circuit Appearance*.
- Carga tu programa en la memoria RAM (clic derecho en la RAM / *Editar Contenidos*). Puedes guardar el contenido de la RAM en un archivo de texto (clic derecho en RAM / *Salvar Imagen*) y volver a cargarlo más adelante (clic derecho en RAM / *Cargar Imagen*).
- En la zona de control de simulación dispones de un botón de **reset**, y un botón para generar la señal de reloj **clk**. También hay visualizadores para observar en todo momento los valores actuales de **PC**, **IR (CO, F y D)** y el estado de la unidad de control.
- Utilizando el control de simulación reinicia la MS y ve ejecutando el programa ciclo a ciclo. En cada ciclo debes saber qué instrucción se está ejecutando, en qué estado está la unidad de control, qué

señales de control se están generando, y qué movimiento de información (acción RTL) se producirá en el cambio de ciclo.

- j) Cuando el programa llegue a la etiqueta **HALT** por primera vez comprueba que el resultado (dato final en la variable **c**) es correcto. Calcula el número de instrucciones ejecutadas y el número de ciclos necesario para la ejecución, sin contar la instrucción de la etiqueta **HALT**.

3 UN PROGRAMA MÁS COMPLEJO

3.1 COMPILACIÓN DEL PROGRAMA

El programa en **lenguaje C** que queremos ejecutar ahora en MS es el siguiente:

```
int vector[]={1,7,12,56,-12,-3,-10,0}; //vector de números a acumular
int suma=0; // variable donde quedará el resultado
int i;      // índice de acceso al vector
int num;    // variable donde obtenemos el número a sumar

i=0;
num=vector[i];
while (num!=0){
    suma=suma+num;
    i=i+1;
    num=vector[i];
}
halt();
```

- a) Traduce el programa a **lenguaje ensamblador**.

```
.data    32
vector:  .dw 1,7,12,56,-12,-3,-10,0; int vector[]={1,7,12,56,-12,-3,-10,0};
suma:    .dw 0                      ; int suma=0;
num:     .rw 1                      ; int num;    la variable i no es necesaria
cero:    .dw 0                      ;
uno:     .dw 1                      ;
const:   .dw 128                    ;

.code
        mov vector, num             ; num=vector[0];
WHILE:                                     ; while (num!=0){
                                           ;
                                           ; suma=suma+num;
        add const, INST             ; i=i+1; (*)
INST:    mov vector, num             ; num=vector[i];
                                           ; }
                                           ;
HALT:                                         ; halt();
                                           ;
.end
```

(*) Esta instrucción modifica a la instrucción que accede al vector, sumando 1 al campo **F** de la instrucción situada en la posición **INST**. Esto se debe a que MS no dispone de un modo de direccionamiento apropiado para recorrer vectores. Por ello, es necesario incrementar el campo **F** de la instrucción que lee el elemento del vector.

Esta técnica se utilizaba en los primeros computadores de los años 1940-60, precisamente por las mismas limitaciones que observamos en MS, denominándose “código automodificable”. En la actualidad sigue usándose para ciertos propósitos, como por ejemplo enlazar código de bibliotecas dinámicas o dificultar la comprensión de un código “valioso” (se llama ofuscar).

- b) Traduce el programa en ensamblador a binario, y obtén el contenido de cada posición de memoria (especificado en hexadecimal).

3.2 SIMULACIÓN Y DEPURACIÓN

- a) Simula el programa ciclo a ciclo con la MS. Presta especial atención a la ejecución de la instrucción que modifica la instrucción **INST**.
- b) Comprueba que el resultado (dato final en la variable **suma**) es correcto. Calcula el número de instrucciones ejecutadas y el número de ciclos necesario para la ejecución hasta llegar por primera vez a la instrucción con la etiqueta **HALT** (sin contar esa instrucción).
- c) Introduce los resultados de los apartados 2.2.j y 3.2.b a través del recurso Moodle “Resultados práctica 6”.